

MARTC: Multi-Agent Retrieval with Task-State Control

Yanlai Wu, Xu Du, Wendong Xiao, and Ying Tang, Fellow, IEEE

Abstract—Large language models can generate fluent responses, but specialized assistance such as tutoring requires grounding in instructional materials, visible task state, and explicit answer criteria. Classical retrieval-augmented generation (RAG) usually treats grounding as a fixed retrieve-then-read step. This control flow is rigid when the same surface question requires different materials, response structure, or evaluation criteria across situations. We propose MARTC (Multi-Agent Retrieval with Task-State Control), a training-free framework that treats RAG as structured prompt construction rather than query-only passage lookup. A supervisory router selects the dominant context requirement (planning, criteria, adaptation, or instructional materials) and decides whether retrieval should run; LLM-backed specialists then prepare labeled context blocks for a fixed generator and one bounded critique/revision pass. On two educational benchmarks, under matched backbone, corpus, reranker, and response cap, MARTC improves rubric- and teaching-point-oriented automatic metrics and reduces token use in most settings. Latency improves on EduBench and for the 27B TutorEval run, but not for the 4B TutorEval run. These results validate context assembly automatically; human pedagogical evaluation remains future work.

I. INTRODUCTION

Large language models (LLMs) are increasingly used as specialized assistants because they can generate explanations, diagnose errors, and provide feedback. However, useful assistance cannot rely only on parametric knowledge. Knowledge-grounded dialogue has long shown the value of external instructional materials [1]; many assistance settings require grounding in curated resources, user-visible context, task history, and evaluation criteria to avoid fluent but misaligned guidance. Educational tutoring is a concrete and demanding instance of this problem: a tutor must coordinate course materials, worked examples, rubrics, learner-visible context, and dialogue history [2], [3].

Classical retrieval-augmented generation (RAG) has therefore become the default grounding mechanism [4]. In classical RAG, the query is embedded, the top- k most similar passages are retrieved, and the LLM generates a response conditioned on those passages.

Yanlai Wu and Ying Tang are with the Department of Electrical and Computer Engineering, Rowan University, Glassboro, NJ 08028 USA (e-mail: wuyanl37@rowan.edu; tang@rowan.edu). Ying Tang is the corresponding author.

Xu Du is with Montclair State University (e-mail: dux3@mail.montclair.edu).

Wendong Xiao is with the School of Automation and Electrical Engineering, University of Science and Technology Beijing, Beijing 100083, P. R. China (e-mail: wdxiao@ustb.edu.cn).

Thus, relevance depends heavily on retrieval quality and query representation [5].

Specialized assistance, such as tutoring, is more complex than factual question answering. The appropriate response may depend on user background, current goal, task history, prior confusion, and evaluation criteria. We call all information available before generation—the learner profile, task, dialogue, rubric, and inline context—the task state. Task-state control is the explicit routing and prompt-assembly mechanism that decides which of these visible constraints should dominate the response. A response must therefore be grounded not only in topically relevant instructional materials but also in the user’s state and task-specific requirements.

Classical RAG can include explicit state text when supplied, but appending user state, dialogue, criteria, and task constraints forces one query embedding to compress topic, level of detail, intent, and response requirements. As constraints grow, search intent can blur. The issue is therefore how a response system should organize constraints before retrieval and generation.

We therefore introduce MARTC for LLM-based tutoring. Its central contribution is a bounded, training-free context controller rather than a new retriever, generator, or open-ended agent loop. By structured prompt construction, we mean separating search intent, learner-state diagnosis, criteria interpretation, and retrieved evidence into labeled blocks before one generator call. A cue-based router selects a prompt-priority regime and retrieval gate; specialist agents fill the corresponding blocks; and one critic checks the draft against the same brief. This design keeps state and criteria available without forcing them into one retrieval embedding.

Contributions. (i) We introduce a transparent controller that combines cue-based task-state routing, specialist prompt construction, and conditional retrieval. (ii) We specify a bounded contract: parallel specialists return structured records, a fixed generator writes one draft, and a critic performs at most one revision. (iii) We validate the controller against Classical and Agentic RAG under paired infrastructure and report quality and resource trade-offs across two backbones.

II. RELATED WORK

RAG controllers. Classical RAG fixes retrieval as the first controller [4]. Self-RAG learns reflection tokens for retrieval and critique [6]; CRAG evaluates retrieved documents and may expand to web search [7]; Adaptive-

RAG learns to choose a reasoning strategy from question complexity [8]; and TARG applies a training-free retrieve-or-skip gate to a no-context draft [9]. These methods control whether, when, or how retrieval is corrected. MARTC instead controls which visible task constraint is represented before retrieval and keeps state and criteria in separate generator blocks. RAG diagnostics identify retrieval and generation failures but do not provide this controller [10].

Agentic retrieval. ReAct, AutoGen, CAMEL, and MetaGPT interleave actions or coordinate roles for broad task execution [11]–[14]. MA-RAG decomposes ambiguous and multi-hop QA into an iterative answer-solving chain [15]. MARTC differs in scope: its agents are bounded context producers, not autonomous answer solvers, and all branches converge on the same generator and one critique pass. Agentic RAG in our experiments retains the specialist scaffold but removes MARTC’s prompt-priority regime and retrieval gate, isolating whether coordination alone explains the result. Educational agents and evaluation. AgentTutor maintains memory and adapts teaching strategy across turns [3]; tutoring benchmarks assess pedagogical response properties [2], [16]–[19]. MARTC is not a complete tutoring policy: it isolates context construction under a fixed generator so memory, dialogue policy, and teaching strategy do not absorb the retrieval effect.

III. METHODOLOGY

Specialized assistance must determine what the user is asking, what state is visible, what criteria apply, and whether external materials are needed. These decisions make context construction a distinct stage before generation. MARTC coordinates that stage through input normalization, supervisor routing, and specialist agents (Fig. 1); Algorithm 1 summarizes inference.

A. Input Normalization

MARTC first converts each request into one normalized input entry:

$$x = (q, o, c, h, r, v). \quad (1)$$

The entry contains only information visible before generation. Here, q is the question or task; o is any answer-option set; c is inline context already provided with the request; h is recent interaction history; r is explicit criteria such as a rubric or required feedback points; and v stores images when present. Any field may be empty. This normalization step is deliberately separated from agent outputs: it only standardizes heterogeneous inputs so the router and specialist agents receive the same interface. This prevents topic, state, criteria, and instructional-material needs from being compressed into a single long retrieval query [3], [20].

B. Supervisor Routing

The supervisory router is responsible for mapping each input sample to a response regime and activating the corresponding specialist agents. Given an input sample, the router examines the visible question, optional context, dialogue history, learner-state information, rubric or criteria text, and task metadata. It then estimates four types of context need: planning, criteria feedback, learner-state adaptation, and instructional-material support.

These needs correspond to four regimes. Planning (PL) is selected when the response requires an ordered structure, such as step sequencing, lesson planning, or exercise design. Criteria Feedback (CF) is selected when the response must be judged against explicit requirements, such as rubrics, scoring rules, or required feedback fields. Adaptive Response (AR) is selected when the same content should be explained differently depending on visible user state, such as grade level, prior confusion, weak points, or preferred explanation style. Instructional Materials (IM) is selected when the response should be supported by external resources, such as textbook chapters, reference documents, or worked examples.

To make routing transparent and training-free, MARTC uses cue scores rather than a learned classifier. It first constructs

$$b(x) = \text{concat}(q, c, r, h), \quad (2)$$

where the four textual fields are normalized and concatenated. Images remain available downstream but are not converted to text. For family $j \in \{\text{im}, \text{cr}, \text{pl}, \text{ad}\}$, let W_j be its cue lexicon and $M_j(x)$ the number of distinct cues occurring in $b(x)$. The score is

$$s_j(x) = \min \left(1, \frac{M_j(x)}{|W_j|} + \delta_j(x) \right), \quad (3)$$

where each phrase contributes at most once. The exact released lexicons are: IM={cite, citation, source, evidence, supporting facts, reference, grounded, retrieval, document, chapter, lecture, verification}; CF={rubric, criterion, criteria, score, feedback, comment}; PL={plan, sequence, schedule, workflow, roadmap, steps, next steps, lesson, curriculum, study schedule, exercise plan}; and AR={user, profile, beginner, expert, personalize, adapt, confused, attempted, hint, misconception, student, explain, teach, tutor}. The explicit adjustments are +0.20 to CF when a rubric is present, up to +0.50 to AR from dialogue length, +0.08 to IM for inline context, and +0.05 to IM for an image.

The fixed thresholds prevent incidental matches from changing behavior. We use $\tau_{\text{mid}} = 0.35$ for CF, AR, and retrieval and $\tau_{\text{plan}} = 0.42$ for PL, whose selection changes response structure. The same values are used for both benchmarks and backbones. A gate sweep over [0.30, 0.55] left aggregate quality unchanged over [0.30, 0.45], so the reported conclusion does not depend on the exact value 0.35 within that interval.

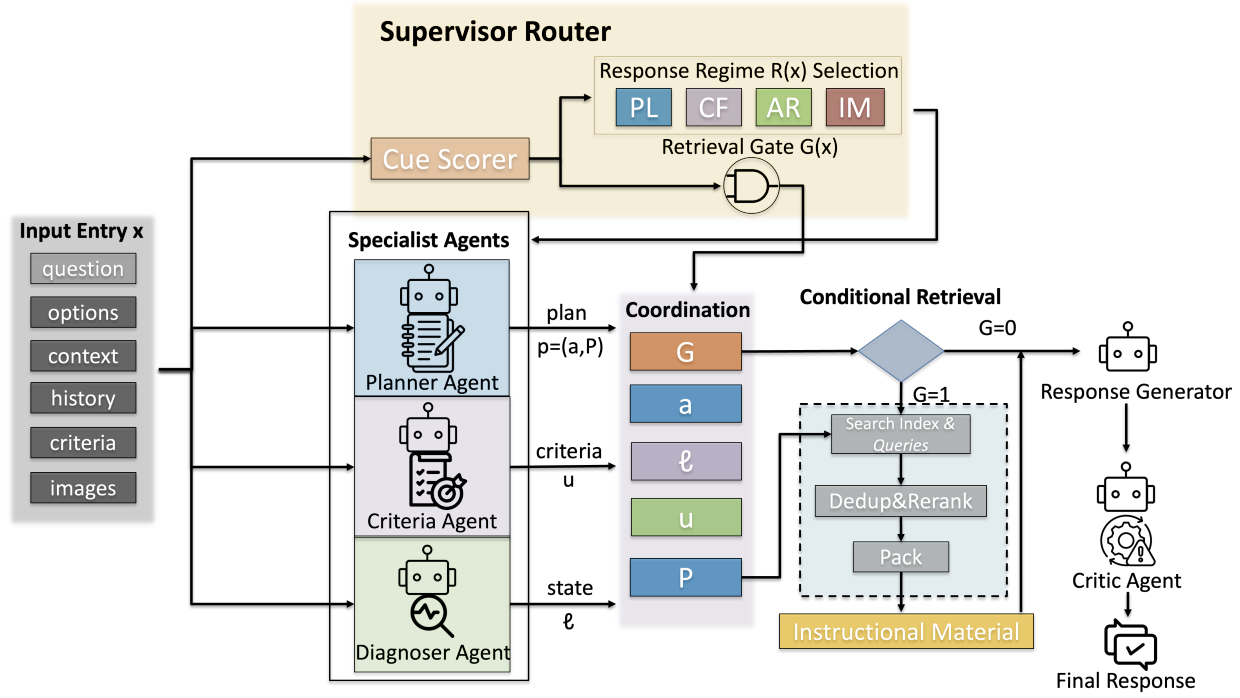


Fig. 1. Main framework of MARTC. A normalized entry $x = (q, o, c, h, r, v)$ is scored by the supervisor router, which selects the response regime $R(x) \in \{PL, CF, AR, IM\}$ and retrieval gate $G(x)$. LLM-based specialist agents then produce structured prompt blocks: planner strategy a and search queries P , criteria summary u , and learner-state summary ℓ . The coordinator merges these blocks with retrieved snippets into instructional-material bundle D when $G(x) = 1$, then sends one augmented prompt to the generator and critic.

Based on these cues and thresholds, the supervisor assigns the response regime as follows:

$$R(x) = \begin{cases} PL, & s_{pl} \geq s_{cr}, s_{pl} \geq s_{ad}, s_{pl} \geq \tau_{plan}, \\ CF, & s_{cr} \geq s_{ad}, s_{cr} \geq \tau_{mid}, \\ AR, & s_{ad} \geq \tau_{mid}, \\ IM, & \text{otherwise.} \end{cases} \quad (4)$$

In addition to selecting the prompt-priority regime, the supervisor makes a binary retrieval decision [6], [8], [9]:

$$G(x) = \begin{cases} 1, & s_{im} \geq \tau_{mid} \text{ or } R(x) = IM, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Thus, retrieval runs when material cues are strong or IM is the default regime. Routing is auditable but lexical: a rubric-bearing grading request without stronger planning/adaptation cues routes to CF, while several dialogue turns showing prior confusion can route to AR; conversely, quoted syllabus language containing “plan,” “schedule,” “roadmap,” and “next steps” can spuriously trigger PL. This false-positive analysis motivates learned calibration and routing-error analysis in future work.

C. Specialist Agents and Coordination

After routing, the planner, criteria agent, and diagnoser construct a response brief in parallel. Each independently returns a structured record, and the coordinator merges the records into one prompt.

The planner returns $p = (a, P)$, where a is a response strategy and P is a short list of scoped retrieval queries.

The criteria agent returns u , summarizing rubrics, grading rules, required fields, or default quality criteria. The diagnoser returns ℓ , summarizing level, goals, misconceptions, prior confusion, or preferred style. MARTC also maintains bundle D for retrieved snippets. Only answer y is user-facing; all other slots are prompt context.

After generation, the critic performs one bounded pass over the brief, materials, and draft to check material use, criteria coverage, and learner-state alignment. It revises once if needed; otherwise it returns the draft.

Each regime changes the brief. PL gives the planner’s structure priority; CF foregrounds evaluation criteria; AR foregrounds ℓ for tone, depth, and scaffolding; and IM uses P to retrieve materials. The final prompt combines the original task, regime, p , ℓ , u , and D when available. This is the structured prompt-construction contract that distinguishes MARTC from an open-ended multi-agent solving chain.

Algorithm 1: MARTC Inference Process

Require: entry $x = (q, o, c, h, r, v)$, index $\mathcal{I}$, thresholds $(\tau_{\text{mid}}, \tau_{\text{plan}})$.
Return: answer y and internal fields (R, G, p, ℓ, u, D) .

```

1:  $b \leftarrow \text{concat}(q, c, r, h)$ ;  $(s_{\text{im}}, s_{\text{cr}}, s_{\text{pl}}, s_{\text{ad}}) \leftarrow \text{CueScores}(b)$ 
2:  $R \leftarrow R(x)$  using Eq. (4);  $G \leftarrow G(x)$  using Eq. (5)
3: parallel do
4:    $p = (a, P) \leftarrow \text{Planner}(x, R, G)$ 
5:    $\ell \leftarrow \text{Diagnoser}(q, h)$ 
6:   if  $r \neq \emptyset$  or  $R = \text{CF}$  then  $u \leftarrow \text{Criteria}(x, R)$ 
7:   else  $u \leftarrow \text{DefaultCriteria}()$ 
8:   end if
9: end parallel
10:  $D \leftarrow \emptyset$ 
11: if  $G = 1$  then
12:    $D \leftarrow \text{Pack}(\text{Rerank}(\text{Dedup}(\text{Search}(\mathcal{I}, P)), q))$ 
13: end if
14:  $y_0 \leftarrow A_{\text{gen}}(\text{Prompt}(q, o, h, p, \ell, u, D, c))$ 
15:  $y \leftarrow \text{CriticOnce}(y_0, q, h, p, \ell, u, D, r)$ 
16: return  $y, (R, G, p, \ell, u, D)$ 

```

IV. VALIDATION EXPERIMENTS

We validate the framework on two public educational benchmarks used as tutoring-focused stress tests. TutorEval [2] pairs science tutoring questions with long STEM chapters and teaching key points; EduBench [17] covers diverse educational scenarios with prompts, meta-data, and reference model predictions that our adapter converts into score and rubric supervision.

The primary validation compares context-assembly methods under matched infrastructure for both Qwen3.5-4B and Qwen3.5-27B-FP8, which checks whether the trends hold across backbone size. Classical RAG retrieves first from the raw standardized prompt; visible learner profile, student answer, dialogue, and criteria text remain available to its generator prompt, so the baseline is not deprived of task context. Its limitation is controller order: retrieval happens before specialist summaries can shape search. Agentic RAG uses the same specialist-coordination scaffold without MARTC’s task-state retrieval control, so it tests whether agent coordination alone explains the gains. MARTC keeps the same backbone and corpus resources, then adds supervisory routing, conditional retrieval, and bounded context assembly. We report quality and cost together because only context construction changes.

A. Implementation Details

All controllers use the same Python framework, corpus, hybrid text index, reranker, generator, critic, cache policy, and metric code; only context assembly differs. Specialists and the generator use the same configured backbone, with deterministic fallbacks only for unavailable or malformed LLM output. MARTC permits bounded local observations and corpus search, without network, file mutation, or open-ended shell tools. SGLang serves the endpoints [21]. Exact model-response caching requires an identical full payload and is recorded in artifacts; latency is therefore cache-aware served wall time, not a cold-start benchmark.

Runs use Qwen3.5-4B and Qwen3.5-27B-FP8 [22], reasoning budget 512, generation temperature 0.15, specialist JSON temperature 0.0, and response cap 900 on benwulab-Lambda-Vector with two NVIDIA RTX 6000

TABLE I

4B quality and resource summary with Agentic comparison.

Data	Metric	RAG	Agentic	MARTC	Δ
TutorEval	Token-F1	0.112	0.090	0.115	+0.004*
	Tutor hit	0.938	0.932	0.957	+0.019**
	Rubric	0.919	0.907	0.944	+0.026**
	Latency (s)	31.90	31.05	39.11	+7.21
	Tokens	3975	5959	3490	-484***
EduBench	EB12D	0.367	0.398	0.398	+0.031***
	Rubric	0.705	0.891	0.891	+0.186***
	EduAlign	0.166	0.130	0.126	-0.040**
	Latency (s)	19.83	23.73	13.90	-5.92***
	Tokens	4020	5224	2226	-1794***

Agentic omits MARTC’s task-state retrieval control. Bold is best among the three methods; lower is better for latency/tokens. Δ and significance compare paired MARTC–RAG only: *** $p < 10^{-4}$, ** $p < 0.01$, * $p < 0.05$.

TABLE II

27B quality and resource summary with Agentic comparison.

Data	Metric	RAG	Agentic	MARTC	Δ
TutorEval	Token-F1	0.264	0.270	0.272	+0.008*
	Tutor hit	0.897	0.902	0.918	+0.021*
	Rubric	0.861	0.873	0.891	+0.030*
	Latency (s)	41.47	48.31	40.65	-0.82**
	Tokens	6681	7200	6730	+49
EduBench	EB12D	0.355	0.389	0.425	+0.070***
	Rubric	0.634	0.748	0.769	+0.135***
	EduAlign	0.541	0.511	0.526	-0.015
	Latency (s)	41.91	50.21	32.09	-9.81***
	Tokens	6105	5829	3717	-2388***

Agentic omits MARTC’s task-state retrieval control. Bold is best among the three methods; lower is better for latency/tokens. Δ and significance compare paired MARTC–RAG only: *** $p < 10^{-4}$, ** $p < 0.01$, * $p < 0.05$.

Ada GPUs. Retrieval packs up to four snippets from at most three planner queries under a 6000-character context cap. For each backbone, TutorEval uses $n=400$ paired samples; EduBench uses $n=328$ unique paired rows after duplicate public IDs in the 400-example slice are collapsed.

All quality scores are automatic proxies, not official human grades. Token-F1 is token-overlap precision/recall F1 against the reference. Rubric coverage averages exact or content-bearing partial matches to required criteria. Tutor-hit assigns full, partial, or zero credit to expert teaching points. EB12D averages 12 normalized checks over scenario adaptation, factual/reasoning behavior, and pedagogical application; EduAlign measures extracted-score agreement with benchmark consensus. Latency is served seconds per sample, and tokens include prompt and completion tokens. These metrics separate overlap, criteria fulfillment, instructional-material use, and resource cost [10], but do not directly measure learning gains.

B. Framework Validation

Tables I and II compare Classical RAG, Agentic RAG, and MARTC. Agentic RAG tests whether specialist coordination alone explains the gains. Significance markers compare paired MARTC–RAG samples using 10,000 paired permutations; 95% intervals use 2,000 paired bootstrap resamples.

C. Discussion

TutorEval gains in tutor-hit and rubric coverage suggest that planner, criteria, and diagnoser slots preserve teaching points and requirements when materials are long. EduBench gains in EB12D and rubric coverage suggest that task-state control helps when requests combine scenario, answer constraints, and feedback style. Agentic RAG is competitive on some quality metrics but often uses more tokens and time because its specialist outputs are not filtered through MARTC’s gate and regime.

EduAlign should be interpreted narrowly because it measures agreement with extracted consensus scores, whereas MARTC constructs criteria-aware responses. The resource results also show a trade-off: scoped queries and gating usually reduce tokens and 27B latency, but 4B TutorEval latency increases by 7.21s and 27B TutorEval uses 49 more tokens. MARTC is therefore not a universal lower-cost guarantee.

D. Limitations

First, automatic proxies do not establish pedagogical helpfulness, trust, affect, or learning gains; no human expert study is reported. Second, the manually specified English cue lexicons can produce lexical false positives, and the gate sweep does not replace labeled routing-accuracy analysis. Third, the benchmarks are English-only and mostly single-turn. Human evaluation, multi-turn outcomes, multilingual calibration, and stronger learned-router baselines are needed before broader tutoring claims.

V. CONCLUSION

MARTC reframes grounding as supervised context construction rather than query-only passage lookup. Its contribution is a training-free controller that separates task-state diagnosis, criteria interpretation, planning, and conditional retrieval before a fixed generator. Under paired automatic validation, this organization improves rubric- and teaching-point-oriented metrics while often reducing resource use. Human and multi-turn evaluation must determine whether these proxy gains translate into better tutoring.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support of Google Academic Research Awards.

References

- [1] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, and J. Weston, “Wizard of Wikipedia: Knowledge-powered conversational agents,” in *Proceedings of the International Conference on Learning Representations*, 2019. [Online]. Available: <https://arxiv.org/abs/1811.01241>
- [2] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, “Language models as science tutors,” *arXiv:2402.11111*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.11111>
- [3] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, “AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems,” *arXiv:2601.04219*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.04219>
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
- [5] N. Thakur, N. Reimers, A. Rüclé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021. [Online]. Available: <https://openreview.net/forum?id=wCu6T5xFjeJ>
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.11511>
- [7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv:2401.15884*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.15884>
- [8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: <https://aclanthology.org/2024.naacl-long.389/>
- [9] Y. Wang, L. Wei, and H. Ling, “Retrieval as a decision: Training-free adaptive gating for efficient RAG,” *arXiv:2511.09803*, 2026. [Online]. Available: <https://arxiv.org/abs/2511.09803>
- [10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, “RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08067>
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv:2308.08155*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [13] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for “mind” exploration of large language model society,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.17760>
- [14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [15] T. Nguyen, P. Chin, and Y.-W. Tai, “MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning,” *arXiv:2505.20096*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.20096>
- [16] K. K. Maurya, K. A. Srivatsa, K. Petukhova, and E. Kochmar, “Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*

- (Volume 1: Long Papers), 2025. [Online]. Available: <https://aclanthology.org/2025.naacl-long.57/>
- [17] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," arXiv:2505.16160, 2025. [Online]. Available: <https://arxiv.org/abs/2505.16160>
 - [18] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.11/>
 - [19] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "TutorBench: A benchmark to assess tutoring capabilities of large language models," arXiv:2510.02663, 2025. [Online]. Available: <https://arxiv.org/abs/2510.02663>
 - [20] C. Borchers and T. Shou, "Can large language models match tutoring system adaptivity? A benchmarking study," in Artificial Intelligence in Education. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: <https://arxiv.org/abs/2504.05570>
 - [21] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
 - [22] Qwen Team, "Qwen3.5: Towards native multimodal agents," Qwen Blog, Feb. 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>